

Preventing Cross-Site Scripting (XSS)

Here's another in our series of articles on simple ways to code safely. This time, we're going to talk about the risks of cross-site scripting and how to prevent them.

To begin, I suppose we ought to discuss what cross-site scripting is in the first place...

There is an inherent, built-in vulnerability in all HTML forms that allows scripts to be injected into web pages and executed without the knowledge of the developer. This is known as cross-site scripting or XSS. There are several forms of cross-site scripting, which I'll detail briefly below, but the primary nature of these attacks takes the following form:

1. A user interacting with the page has a place where he or she can enter some data that the page will use. These locations can include:
 - a. Input fields on a form
 - b. The URL query string
 - c. Post data
2. The user enters HTML script tags with some script content
3. The browser (or server-side handler) executes the code

Why would code inserted by a user be executed?

Browsers (and any other script-running processes) read the DOM (document object model). Having done so, each time they encounter a `<script>...</script>` pair, they execute the code contained therein. There is no safeguard to tell the browser that a script wasn't part of its original source code.

What kinds of XSS attacks exist?

- Persistent XSS
 - In a persistent XSS attack, the malicious user creates an attack that occurs every time a page is viewed or every time a certain step is taken from that page.
 - The key here is that the malicious code is stored and executed more than once.
 - For example:
 - A user might know that a certain field on the form is stored in the database for future use. They may enter a script that would execute every time that database item was retrieved and added to the DOM.
 - Or a user might be on a page that functions as a guestbook. Instead of entering a name, he or she would enter a script as the value, which, once added to the DOM, would execute every time someone opened the page.
- Reflected XSS
 - In a reflected attack, the script content is part of the malicious user's request to the web page.
 - The prime example of this sort of attack is when components of the query string are used immediately on page load.
 - These sorts of attacks are often provided as hyperlinks in an email, where the malicious user isn't trying to attack the vulnerable site, but is instead simply exploiting its vulnerability to ensure that a script runs in the user's browser.

- o For example:
 - A user might be directed by the malicious link to a “search results” page. When the page loads, in addition to the search results, the search criteria (which are part of the URL) are displayed on the page. Since the criteria are added to the DOM for display, any script embedded in the URL would execute.
 - Or a user might modify the search criteria to obtain additional information beyond what the search provides and display it. In this instance, the user is only interested in the script running in his or her own browser, but is exploiting the same vulnerability.

So, where does the vulnerability occur, and how do we work to prevent it?

This vulnerability occurs whenever any content coming from outside the page is used without sanitizing it to ensure that it does not contain control characters that could execute code.

Preventing the vulnerability entails understanding what control characters need to be handled and how to escape them.

Well, where does this impact us?

We have a large number of e-forms where data entered by the user is either used immediately or stored into a database without scrubbing. On any of these, a person could store malicious script code.

One specific example would be the ubiquitous “add comments” scripts I see on e-forms, where the content of one field is added verbatim into a textarea on the form (adding it to the DOM), which could result in malicious code executing every time a user retrieved and viewed that document.

OK. I’m convinced. How do I prevent this?

Turn the page and read on, my friend...

Code Examples:

Here is an example of a web page with two XSS vulnerabilities (see the embedded zip file at the end for complete code)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Cross-Site Scripting (XSS) Example [Vulnerable]</title>
  <style>
    ...
  </style>
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.4/jquery.min.js"></script>
  <script type="text/javascript">
    var urlParams = new URLSearchParams(window.location.search);
    $(document).ready(function () {
      if (urlParams.has("myData")) {
        $("#myData").html(urlParams.get("myData"));
      }

      $(".input").on("blur", function () {
        $("#resultField").html($(this).val());
      });
    });
  </script>
</head>
<body>
  <form method="post" action="#">
    <div>
      On this page, there is a vulnerability to cross-site scripting (XSS).
    </div>
    <div>
      The 'myData' component of the query string contains [<span id="myData">nothing</span>]
    </div>
    <div>
      <input type="text" name="vulnerableField" id="vulnerableField" value=""/>
    </div>
    <div>
      <textarea name="resultField" id="resultField"></textarea>
    </div>
  </form>
</body>
</html>
```

As you can see from the highlighted lines, we are using data from a form field and from the URL query string parameters. We are not performing any sanitization of the data. As a result, when a user adds safe data such as this:

This is a safe value

The textarea is updated with the safe text.

This is a safe value

Likewise, if safe data enters via the URL, like this:

`/Vulnerable.html?myData=something%20safe`

The page is safely updated with the expected data...

The 'myData' component of the query string contains [something safe]

But, if a malicious user enters something like this:

```
<script>alert("I inserted JavaScript!");</script>
```

Here's the result:



Likewise, if a script is injected via the URL like this:

`/Vulnerable.html?myData=<script>alert(%27JavaScript%20Injected!%27);</script>`

We see this result



No, in these examples, I did not do anything really malicious, and I have no intention of including real malice as a lesson topic. If you want to be a villain, learn that yourself. However, this example does serve to illustrate how easy it is to cause unexpected code to execute on the page.

Fortunately, preventing this is relatively easy, if a little tedious. Doing it requires understanding that in HTML, we have entities that function as character escapes for characters that would otherwise have control values in HTML. If we can prevent the `<script>` tag (which informs the browser that a new script element with active code is starting) from being interpreted, then its content would not be executed.

The important characters to escape in this way are these:

Character	HTML Escape
<	<
>	>
"	"
'	'
&	&

Enter the “sanitize” function

```
function Sanitize(value) {  
    value = value.replace(/</g, "&lt;");  
    .replace(/>/g, "&gt;");  
    .replace(/&/g, "&amp;");  
    .replace(/'/g, "&#39;");  
    .replace(/"/g, "&quot;");  
    return value;  
}
```

Here, all we’re doing is modifying the user input to remove any control characters and insert printable entities in their places.

Now we modify our existing script elements thus:

```
if (urlParams.has("myData")) {  
    var value = Sanitize(urlParams.get("myData"));  
    $("#myData").html(value);  
}  
  
$("#input").on("blur", function () {  
    var value = Sanitize($(this).val());  
    $("#resultField").html(value);  
});
```

When safe data occurs, this functions exactly as the vulnerable example did, but when unsafe data is entered, the results are entirely different. Using the same examples as above, we now see this...

The 'myData' component of the query string contains `[<script>alert('JavaScript Injected!');</script>]`

And this...

```
&lt;script&gt;alert("I inserted JavaScript!");&lt;/script&gt;
```

That's all it takes to make the page safe, but the important thing is that you must do this everywhere, every time. Even one lapse where you utilize user data without sanitizing will re-introduce this vulnerability into your page.

Here are the complete example HTML pages



HtmExamples.zip

Be safe, and happy coding!